

Usaría **Heapsort con HEAPIFY iterativo**, suponiendo que los datos de una noche caben en un arreglo en memoria y que no se exige estabilidad.

1. **Funcionamiento.** Construye un montículo máximo sobre el arreglo. Después intercambia la raíz con la última posición del montículo, reduce su tamaño y restaura la propiedad de montículo. Como la raíz contiene el máximo pendiente, cada extracción coloca un elemento en su posición definitiva.
2. **Memoria y recursión.** Trabaja sobre el mismo arreglo y utiliza únicamente un número fijo de índices y una variable para intercambios: la memoria adicional pertenece a $O(1)$. Al implementar HEAPIFY mediante un ciclo, no necesita recursión ni una pila de llamadas.
3. **Tiempo garantizado.** Un montículo de m elementos es un árbol binario completo de altura $\lceil \log_2 m \rceil$. Cada restauración desciende, como máximo, esa cantidad de niveles y realiza trabajo constante por nivel; por tanto, cuesta $O(\log m)$.

La construcción realiza a lo sumo n restauraciones y el ordenamiento realiza $n - 1$ extracciones. Como cada montículo tiene a lo sumo n elementos, el tiempo total pertenece a $O(n \log n)$ en el peor caso, y esta misma cota superior vale para el promedio. Esta garantía permite limitar el cómputo nocturno independientemente del orden inicial.

4. **Comparación.**

- **Quicksort:** con pivotes aleatorios tiene tiempo esperado $\Theta(n \log n)$, pero una sucesión de particiones de tamaños 0 y $m - 1$ produce tiempo cuadrático. Su implementación recursiva usual también puede acumular $\Theta(n)$ llamadas.
- **Mergesort:** garantiza tiempo $\Theta(n \log n)$, pero su implementación habitual sobre arreglos requiere memoria auxiliar $\Theta(n)$ y, si es recursiva, una pila de profundidad $\Theta(\log n)$.
- **Insertion sort:** usa memoria adicional $\Theta(1)$ y puede aprovechar datos casi ordenados. Sin embargo, con datos distintos en orden inverso desplaza $\sum_{i=2}^n (i - 1) = n(n - 1)/2$ elementos, por lo que su peor caso es cuadrático.

Conclusión. Heapsort iterativo combina memoria adicional constante, ausencia de recursión y tiempo $O(n \log n)$ garantizado. Es adecuado para los recursos limitados del explorador; no es estable, por lo que esta elección supone que no se necesita conservar el orden original entre datos con claves iguales. ■